Using Go & Clojure with Babashka pods to create Conjtest

Exploring a Policy as Code tool made using Clojure, Babashka, Babashka pods, and Go.

Whoami

```
$ whoami
ilmo.raunio
```

- Currently at SOK in S-ID team (some Clojure services, mainly TypeScript)
  - As security champion, do a lot of threat modeling & think about prevention mechanisms
  - Teach people how to do Site Reliability Engineering (SRE), improve observability
  - Gospel about Clojure and try to make people fall in love with it
- Ex-Metosin
  - Maintainer of metosin/oksa, a small GraphQL query library
- Father of two kids & passionate Japanese cuisine enthusiast

Policy as Code

> Policy as code gives you an automated way to check, in seconds, if your IT and business stakeholders' requirements are being followed — in every deployment. **A policy as code framework includes a policy coding language that can be tested, peer reviewed, versioned, automated, and re-used much like application or infrastructure code.**

Emphasis mine.

https://www.hashicorp.com/en/blog/policy-as-code-explained

Policy Language

OPA is purpose built for policy evaluation and uses its declarative language Rego to reason about structured data like API requests, infrastructure-as-code files, and configuration data. Rego lets you express desired rules and decisions as code, and is designed to be easy to read and write while being optimized for fast policy evaluation.

Rego queries are assertions on data that can be used to define policies and make decisions about whether data violates the expected state of your system. Rego was inspired by Datalog and extends it to support structured document models such as JSON.

Why use Rego?

Use Rego for defining policy that is easy to read and write.

Rego focuses on providing powerful support for referencing nested documents and ensuring that queries are correct and unambiguous.

Rego is declarative so policy authors can focus on what queries should return rather than how queries should be executed. These queries are simpler and more concise than the equivalent in an imperative language.

Like other applications which support declarative query languages, OPA is able to optimize queries to improve performance.

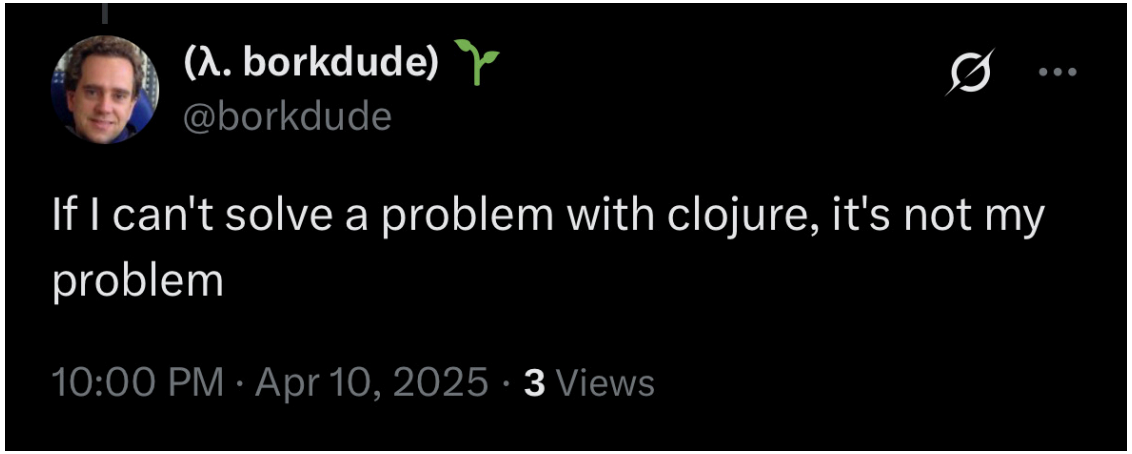
```
1  package main
2
3  import data.util as util
4
5  name = input.metadata.name
6
7  deny[msg] {
8    allowlist := [
9      "cdn",
10   ]
11   input.kind == "Ingress"
12   input.metadata.annotations["nginx.ingress.kubernetes.io/cors-allow-origin"]
     == "*"
13   not util.exists_in_list(name, allowlist)
14   msg := sprintf("CORS is too permissive (nginx.ingress.kubernetes.io/
     cors-allow-origin)', name: %s, actual: '%s'", [name,
     input.metadata.annotations["nginx.ingress.kubernetes.io/cors-allow-origin"]])
15 }
```

Conftest

Conftest is a utility to help you write tests against structured configuration data. For instance, you could write tests for your Kubernetes configurations, Tekton pipeline definitions, Terraform code, Serverless configs or any other structured data.

Conftest relies on the Rego language from Open Policy Agent for writing policies. If you're unsure what exactly a policy is, or unfamiliar with the Rego policy language, the Policy Language documentation provided by the Open Policy Agent documentation site is a great resource to read.

Initial requirements



- Define policy-as-code using Clojure
- Support for many config formats: CUE, CycloneDX, Dockerfile, EDN, .env, HCL, HOCON, Ignore files, INI, JSON, Jsonnet, Properties, SPDX, TextProto, TOML, VCL, XML, YAML

Clojure doesn't have this level of support, while Go does (collated via projects like Conftest)

Enter Babashka

A short dive into Babashka & Babashka pod protocol

What's Babashka?

- Native Clojure interpreter for scripting with fast startup (uses GraalVM under the hood)
- Allows you to use Clojure in places where you would be using bash otherwise
- Maintained by borkdude (Michiel Borkent)

```
bb -e '(str (fs/cwd))'
; => "/Users/ilmo.raunio"

bb -e '(->> (fs/list-dir ".")
         (filter fs/directory?)
         (map fs/normalize)
         (map str)
         (filter (complement #(clojure.string/starts-with? % "."))))'
; => ("Devel" "Music" "go" "Pictures" "Desktop" "Library" "epic-workshops" "Public" "Movies" "Documents" "Downloads")
```

What are Babashka pods?

- Programs that can be used as Clojure libraries inside Babashka.
- Pods can be built in Clojure, but also in languages that don't run on the JVM.
  - Eg: Go, Haskell, Rust.
- Pods can be created independently from pod clients. Any program can be invoked as a pod as long as it implements the pod protocol.

Babashka pod protocol

- Exchange of messages between pod client and the pod which happen in the bencode format
- Protocol uses opcodes like "describe", "invoke", "shutdown", etc.
- Additionally, payloads like args (arguments) or value (a function return value) are encoded in either EDN, JSON or Transit JSON

```
(bencode/write-bencode System/out {"op" "describe"})
; => d2:op8:describee
```

```
;; expected pod response
{"format" "json"
 "namespaces"
 [{"name" "pod.lispyclouds.sqlite"
   "vars" [{"name" "execute!"}]}]
 "ops" {"shutdown" {}}}
```

https://github.com/babashka/pods#the-protocol

Bencode quick intro

- Supports the following types of values:
  - byte strings
  - integers
  - lists (arrays)
  - dictionaries (associative arrays)
- Babashka pods uses bencode to encode the message from the pod client to the pod, and vice versa

Pod invocation example

```
user=> (bencode/write-bencode System/out {"op" "invoke"
                                          "id" "fa18772a-7159-47fb-96b8-7426d4c8891e"
                                          "var" "pod.ilmoraunio.conftest/parse"
                                          "args" "[\"~#list\",[\"bb.edn\"]]"})
; => d4:args21:["~#list",["bb.edn"]]2:id36:fa18772a-7159-47fb-96b8-7426d4c8891e2:op6:invoke3:var29:pod.ilmoraunio.conftest/parseenil
```

```
$ ./pod-ilmoraunio-conftest
d4:args21:["~#list",["bb.edn"]]2:id36:fa18772a-7159-47fb-96b8-7426d4c8891e2:op6:invoke3:var29:pod.ilmoraunio.conftest/parseenil

d2:id36:fa18772a-7159-47fb-96b8-7426d4c8891e6:statusl4:donee5:value438:["^
","bb.edn",["^ ",":tasks",["^ ","test",["^
",":requires",[["conjtest.bb.main",":as","main"]],":task",["main/test","*command-line-args*"]],"parse",["^
","^3",[["conjtest.bb.main",":as","main"]],"^4",["main/parse","*command-line-args*"]]],":paths",["src","pod-ilmoraunio-conjtest/src","resources/"],":deps",["^
","org.conjtest/conjtest-clj",["^ ",":mvn/version","0.4.0"]],":pods",["^
","ilmoraunio/conftest",["^ ",":version","0.1.1"]]]]e
```

Decoding:

```
user=> (clojure.pprint/pprint (bencode->clj "d2:id36:fa18772a-7159-47fb-96b8-7426d4c8891e6:statusl4:donee5:value438:[\"^ \'
```

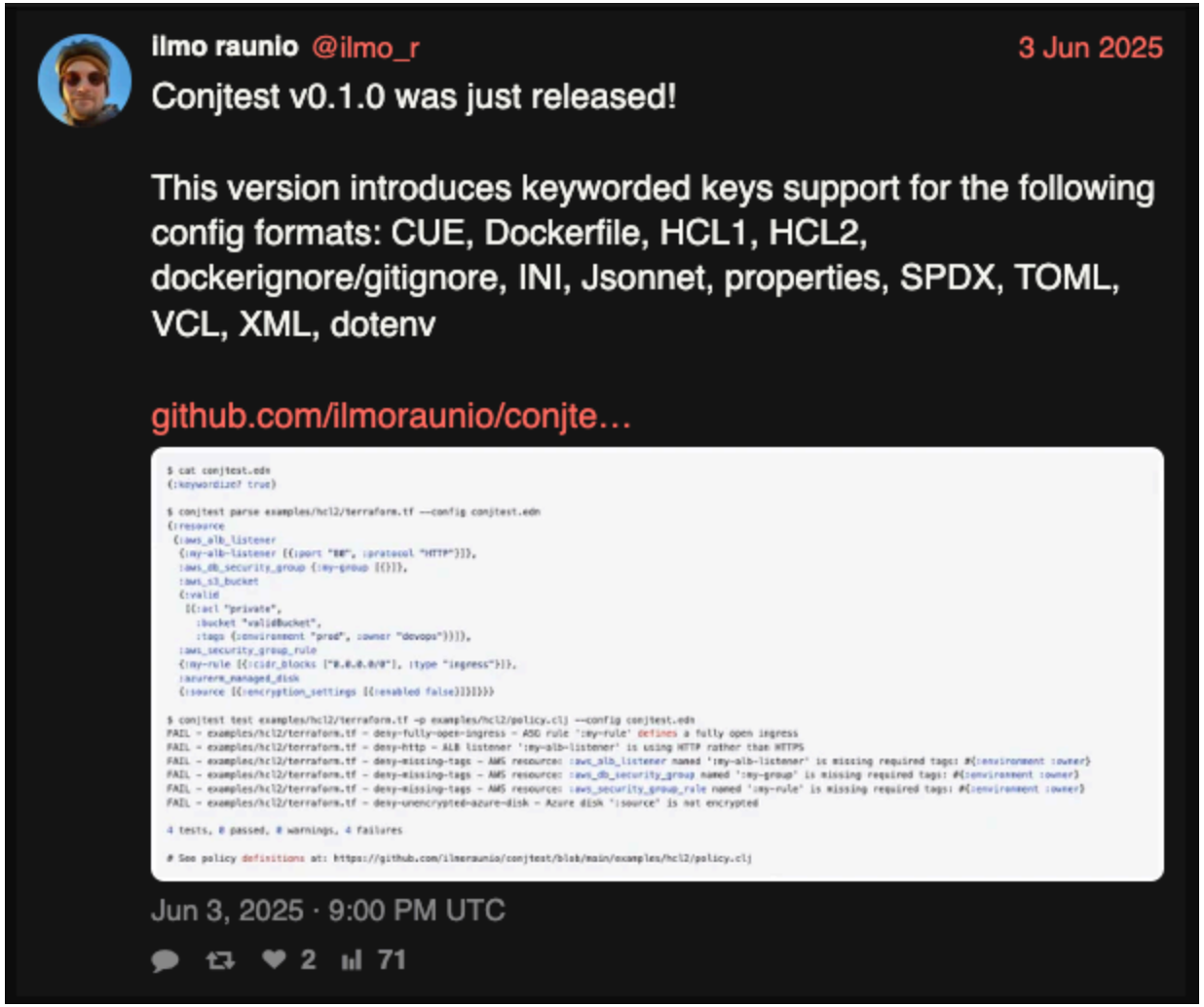
```
{"id" "fa18772a-7159-47fb-96b8-7426d4c8891e",
 "status" ["done"],
 "value"
 "[\"^ \",\"bb.edn\",[\"^ \",\":tasks\",[\"^ \",\"test\",[\"^ \",\":requires\",[[\"conjtest.bb.main\",\":as\",\"main\"]],\'
```

Pod registry

https://github.com/babashka/pod-registry

Pod examples

- Go: https://github.com/babashka/pod-babashka-go-sqlite3
- Rust: https://github.com/babashka/pod-babashka-filewatcher
- Haskell: https://github.com/rorokimdim/stash



Thanks to Babashka, Babashka pods, and the Go ecosystem, this idea became possible.

Demo time

https://github.com/ilmoraunio/conjtest/tree/main?tab=readme-ov-file#quickstart

- conjtest (x86 & arm64 binaries)
- conjtest-clj, pod-ilmoraunio-conjtest, pod-ilmoraunio-conftest are published separately as libraries/pods
    - allows for custom scripting via Babashka, can even BYOParser

```
// pod-ilmoraunio-conftest (main.go)

import (
    …
    "github.com/open-policy-agent/conftest/parser"
    …
)

func processMessage(message *babashka.Message) {
    switch message.Op {
    case "describe":
        babashka.WriteDescribeResponse(
            &babashka.DescribeResponse{
                Format: "transit+json",
                Namespaces: []babashka.Namespace{
                    {
                        Name: "pod.ilmoraunio.conftest",
                        Vars: []babashka.Var{
                            {
                                Name: "parse",
                            },
                            {
                                Name: "parse-as",
                            },
                        },
                    },
                },
            })
    case "invoke":
        switch message.Var {
        case "pod.ilmoraunio.conftest/parse":
            args, err := parseArgs(message.Args)
            if err != nil {
                babashka.WriteErrorResponse(message, err)
                return
            }

            configs, err := parser.ParseConfigurations(args)
            if err != nil {
                babashka.WriteErrorResponse(message, err)
                return
            } else {
                respond(message, configs)
            }
…
```

```
;; pod-ilmoraunio-conjtest

(defn -main []
  (loop []
    (let [message (try (read-bencode stdin)
                       (catch EOFException _
                         ::EOF))]
      (debug "message" message)
      (when-not (identical? ::EOF message)
        (let [op (-> message (get "op") bytes->string keyword)
              id (or (some-> message (get "id") bytes->string)
                     "unknown")]
          (case op
            :describe (do
                        (debug "got :describe")
                        (debug "responding with" (pr-str @describe-map))
                        (write-bencode stdout @describe-map)
                        (recur))
            :invoke (do
                      (debug "got :invoke")
                      (try
                        (let [response (dispatch message)] ; <-- ENHANCE
                          (debug "responding with" (pr-str response))
                          (write-bencode stdout response))
                        (catch Throwable e
                          (->> e (error-response id) (write-bencode stdout))))
                      (recur))
            :shutdown (do
                        (debug "shutting down")
                        (System/exit 0))
            (do
              (let [response {"ex-message" "Unknown op"
                              "ex-data"    (pr-str {:op op})
                              "id"         id
                              "status"     ["done" "error"]}]
                (write-bencode stdout response))
              (recur)))))))))
```

```
;; pod-ilmoraunio-conjtest

(defn dispatch
  [message]
  (debug message)
  (let [id (bytes->string (get message "id"))
        _ (debug "id" id)
        var (bytes->string (get message "var"))
        _ (debug "var" var)
        args (-> (get message "args") bytes->string edn/read-string)
        _ (debug "args" args)
        value (case var
                "pod-ilmoraunio-conjtest.api/parse" (apply api/parse args)
                "pod-ilmoraunio-conjtest.api/parse-as" (apply api/parse-as args)
                "pod-ilmoraunio-conjtest.api/parse-go" (apply api/parse-go args)
                "pod-ilmoraunio-conjtest.api/parse-go-as" (apply api/parse-go-as args)
                "pod-ilmoraunio-conjtest.api/parse*" (apply api/parse* args)
                "pod-ilmoraunio-conjtest.api/parse-as*" (apply api/parse-as* args)
                "pod-ilmoraunio-conjtest.api/parse-go*" (apply api/parse-go* args)
                "pod-ilmoraunio-conjtest.api/parse-go-as*" (apply api/parse-go-as* args))
        _ (debug "value" value)]
    {"value" (pr-str value)
     "id" id
     "status" ["done"]}))
```

Conjtest features

Rule types

```
;; Allow rule
(defn allow-my-absolute-bare-rule
  [input]
  (and (= "v1" (:apiVersion input))
       (= "Service" (:kind input))
       (= 80 (-> input :spec :ports first :port))))

;; Deny rule
(defn deny-my-absolute-bare-rule
  [input]
  (and (= "v1" (:apiVersion input))
       (= "Service" (:kind input))
       (not= 80 (-> input :spec :ports first :port))))

;; Warn rule
(defn warn-my-absolute-bare-rule
  [input]
  (and (= "v1" (:apiVersion input))
       (= "Service" (:kind input))
       (not= 80 (-> input :spec :ports first :port))))
```

Custom error message

Function may also return a failure message (string) to indicate a violation.

```
(defn deny-should-not-run-as-root
  [input]
  (let [name (-> input :metadata :name)]
    (when (and (deployment? input)
               (true? (get-in input [:spec :template :spec :securityContext :runAsNonRoot])))
      (format "Containers must not run as root in Deployment \"%s\"" name))))
```

Multiple error messages

Function may also return a non-empty collection of error messages.

```
(defn deny-missing-tags
  [input]
  (for [[resource-type resources] (:resource input)
        [resource-name resource-definitions] resources
        resource-definition resource-definitions
        :let [tags (into #{} (keys (:tags resource-definition)))
              missing-tags (clojure.set/difference (clojure.set/union tags required-tags) tags)]
        :when (and (clojure.string/starts-with? (name resource-type) "aws_") (pos? (count missing-tags)))]
    (format "AWS resource: %s named '%s' is missing required tags: %s" resource-type resource-name missing-tags)))
```

```
$ conjtest test examples/hcl2/terraform.tf -p examples/hcl2/policy.clj
FAIL - examples/hcl2/terraform.tf - deny-missing-tags - AWS resource: :aws_db_security_group named ':my-group' is missing required tags: #{:environment :owner}
FAIL - examples/hcl2/terraform.tf - deny-missing-tags - AWS resource: :aws_security_group_rule named ':my-rule' is missing required tags: #{:environment :owner}
FAIL - examples/hcl2/terraform.tf - deny-missing-tags - AWS resource: :aws_alb_listener named ':my-alb-listener' is missing required tags: #{:environment :owner}

1 tests, 0 passed, 0 warnings, 1 failures
```

Declarative policies

Malli schemas are evaluated using `malli.core/validate` after which they are processed for any errors using `malli.core/explain` and `malli.error/humanize`.

```
(ns policy)

(def allow-declarative-example
  [:map
   [:kind [:= "Deployment"]]
   [:metadata
    [:map
     [:name :string]]]
   [:spec
    [:map
     [:selector
      [:map
       [:matchLabels
        [:map
         [:app :string]
         [:release :string]]]]]
     [:template
      [:map
       [:spec
        [:map
         [:securityContext {:optional true}
          [:map
           [:runAsNonRoot [:not= {:error/message "Containers must not run as root"}
                           true]]]]]]]]]])
```

DIY - Using conjtest through Babashka as a dependency

```
;; bb.edn
{:paths ["."]
 :deps {org.conjtest/conjtest-clj {:mvn/version "0.4.0"}}
 :pods {ilmoraunio/conjtest {:version "0.1.2"}}
 :tasks
 {:requires ([main])
  test (apply main/test *command-line-args*)}}
```

```
;; main.clj
(ns main
  (:require [conjtest.core :as conjtest]
            [pod-ilmoraunio-conjtest.api :as parser]
            [policy]))

(defn test
  [& args]
  (let [inputs (apply parser/parse args)]
    (let [{:keys [summary-report failure-report]}
          (conjtest/test inputs 'policy)]
      (cond
        failure-report (do (println failure-report) (System/exit 1))
        summary-report (do (println summary-report) (System/exit 0))))))
```

```
bb test sample_config.edn
```

Debugging

- `println`, `prn`, `print`
- Tracing (`conjtest test … --trace`)
- REPL (`conjtest repl`)

Tracing

```
$ conjtest test test-resources/invalid.yaml --policy test-resources/conjtest/example_deny_rules.clj --trace

Policy argument(s): test-resources/conjtest/example_deny_rules.clj
Filenames parsed: test-resources/invalid.yaml
Policies used: test-resources/conjtest/example_deny_rules.clj
TRACE:
---
...
Rule name: deny-my-rule
Input file: test-resources/invalid.yaml
Parsed input: {:apiVersion "v1",
 :kind "Service",
 :metadata {:name "hello-kubernetes"},
 :spec
 {:type "LoadBalancer",
  :ports ({:port 81, :targetPort 8080}),
  :selector {:app "bad-hello-kubernetes"}}}
Result: true
...
FAIL - test-resources/invalid.yaml - deny-malli-rule - :conjtest/rule-validation-failed
FAIL - test-resources/invalid.yaml - deny-my-absolute-bare-rule - :conjtest/rule-validation-failed
FAIL - test-resources/invalid.yaml - deny-my-bare-rule - port should be 80
FAIL - test-resources/invalid.yaml - deny-my-rule - port should be 80
FAIL - test-resources/invalid.yaml - differently-named-deny-rule - port should be 80

5 tests, 0 passed, 0 warnings, 5 failures
```

REPL

```
$ conjtest repl
Started nREPL server at 0.0.0.0:1667
```

```
$ clj -Sdeps '{:deps {nrepl/nrepl {:mvn/version "0.5.3"}}}' -m nrepl.cmdline -c --host 127.0.0.1 --port 1667
...
user=> (do (require '[conjtest.core :as conjtest]) (require '[pod-ilmoraunio-conjtest.api :as api]))
nil
user=> (defn deny-my-policy
         [input]
         (when ((into #{} (:paths input)) "evil-dir")
           "evil-dir found!"))
#'user/deny-my-policy
user=> (deny-my-policy (first (vals (api/parse "deps.edn"))))
nil
user=> (deny-my-policy (update (first (vals (api/parse "deps.edn")))
                               :paths
                               conj
                               "evil-dir"))
"evil-dir found!"
user=> (conjtest/test [(update (first (vals (api/parse "deps.edn")))
                               :paths
                               conj
                               "evil-dir")] #'deny-my-policy)
{:summary {:total 1, :passed 0, :warnings 0, :failures 1}, :failure-report "FAIL - null - deny-my-policy - evil-dir found!\
```

More features covered in the docs

Highlights

- Keyworded keys support
- Metadata support (rename functions & define top-level error message)
- Use Go/Conftest parsers only (useful for porting rules to Clojure)

Takeaways

Babashka ecosystem

- Babashka
  - Batteries included (special mentions: babashka.cli, babashka.fs)
  - Whatever isn't documented, probably exists as code in a repo that borkdude wrote years ago
- Babashka pods
  - Was hard to get started at first
  - Luckily, code existed in many repos that borkdude had written years ago
  - Once setup was done, development became easier
- Babashka pod registry
  - Fast reviews (❤️ borkdude)
  - Current setup of releasing two pods produces a lot of handiwork (but maybe it's worth it)
- Had so much trouble with Windows binary
  - Skill issue probably
  - Was a good idea to scope it out from the first release
  - Maybe revisit if somebody really wants it

CLJ<->Go Interop

- Parser results can differ greatly between Clojure & Go ecosystems, eg. EDN or XML

```
→  conjtest git:(meetup) ✗ conjtest parse deps.edn
{:deps
 {org.clojure/clojure #:mvn{:version "1.12.0"},
  ilmoraunio/conjtest-clj #:mvn{:version "0.4.0"}}}
→  conjtest git:(meetup) ✗ conjtest parse -go deps.edn
{":deps"
 {"org.clojure/clojure" {":mvn/version" "1.12.0"},
  "ilmoraunio/conjtest-clj" {":mvn/version" "0.4.0"}}}
```

- One parser in particular didn't support explicit parameters but requires command-line arguments which we can't pass through Babashka pod protocol
- Open question: How to resolve Go panics in a Babashka pod?

Performance

- tl;dr: Not great, terrible!

```
$ conjtest git:(meetup) ✗ hyperfine 'conjtest parse examples/hcl2/terraform.tf'
Benchmark 1: conjtest parse examples/hcl2/terraform.tf
  Time (mean ± σ):      98.1 ms ±   1.4 ms    [User: 78.7 ms, System: 13.3 ms]
  Range (min … max):    95.5 ms … 101.3 ms    28 runs

$ hyperfine -i 'conjtest test examples/hcl2/terraform.tf -p examples/hcl2/policy.clj --config conjtest.edn'
Benchmark 1: conjtest test examples/hcl2/terraform.tf -p examples/hcl2/policy.clj --config conjtest.edn
  Time (mean ± σ):      97.3 ms ±   1.5 ms    [User: 78.4 ms, System: 13.5 ms]
  Range (min … max):    94.0 ms … 100.5 ms    28 runs
```

```
# Successful optimization attempt: 40ms shaved after wiring Babashka to Babashka pod directly using `:paths`

$ hyperfine './conjtest parse examples/**/*.{yaml,yml}'
Benchmark 1: ./conjtest parse examples/**/*.{yaml,yml}
  Time (mean ± σ):     148.7 ms ±   5.8 ms    [User: 25.4 ms, System: 18.6 ms]
  Range (min … max):   145.5 ms … 170.5 ms    17 runs

# Post-optimization (https://gist.github.com/ilmoraunio/54fae7ebb05c9768778057e002cc50d9)
  Time (mean ± σ):     108.2 ms ±   4.0 ms    [User: 24.2 ms, System: 47.2 ms]
  Range (min … max):   106.1 ms … 125.7 ms    23 runs
```

Could it be any better? Ideas/Contributions/PRs welcome!

Summing up

- It's possible to use Go libraries from Clojure using Babashka pods.
  - Won't be fast necessarily.
- You can write policy-as-code tooling (and rules) using Clojure.
- You could probably write other data-oriented SRE tooling using Clojure as well.
- Babashka is awesome.

Gratitude 🌼

- borkdude for Babashka, Sci, pods, registry
- Conftest creators & contributors
- Kimmo Koskinen (for introducing pods to my brain years ago)
- Miikka Koskinen (for sparring)
- Helsinki FP meetup community

Discussion & questions

https://github.com/ilmoraunio/conjtest (MIT, PRs welcome!)

https://user-guide.conjtest.org

#conjtest @ Clojurians Slack